

Java String Programs Using Only Core Java (No Collections)

1. Identifying Unique Characters in a String

```
public class UniqueCharacters {
    public static void main(String[] args) {
        String input = "programming";
        for (int i = 0; i < input.length(); i++) {
            char c = input.charAt(i);
            boolean isUnique = true;
            for (int j = 0; j < input.length(); j++) {
                if (i != j && c == input.charAt(j)) {
                    isUnique = false;
                    break;
                }
            }
            if (isUnique) {
                System.out.print(c + " ");
            }
        }
    }
}
```

Explanation: This program checks each character against all others to find and print unique characters without using collections.

2. Identifying Duplicate Characters in a String

```
public class DuplicateCharacters {
    public static void main(String[] args) {
        String input = "programming";
        boolean[] printed = new boolean[256];
        for (int i = 0; i < input.length(); i++) {
            char c = input.charAt(i);
            if (printed[c]) continue;
            for (int j = i + 1; j < input.length(); j++) {
                if (c == input.charAt(j)) {
                    System.out.print(c + " ");
                    printed[c] = true;
                    break;
                }
            }
        }
    }
}
```

Explanation: This program uses a boolean array to track printed duplicates and compares characters using nested loops.

3. Reverse a String

```
public class ReverseString {
    public static void main(String[] args) {
        String input = "hello";
```

```

        String reversed = "";
        for (int i = input.length() - 1; i >= 0; i--) {
            reversed += input.charAt(i);
        }
        System.out.println("Reversed: " + reversed);
    }
}

```

Explanation: This program reverses a string by appending characters from end to start using a loop.

4. Check if a String is Palindrome

```

public class PalindromeCheck {
    public static void main(String[] args) {
        String input = "madam";
        boolean isPalindrome = true;
        for (int i = 0; i < input.length() / 2; i++) {
            if (input.charAt(i) != input.charAt(input.length() - 1 - i)) {
                isPalindrome = false;
                break;
            }
        }
        System.out.println(isPalindrome ? "Palindrome" : "Not a palindrome");
    }
}

```

Explanation: This program compares characters from both ends to check if the string is a palindrome.

5. Count Vowels and Consonants

```

public class VowelConsonantCount {
    public static void main(String[] args) {
        String input = "hello world";
        int vowels = 0, consonants = 0;
        for (int i = 0; i < input.length(); i++) {
            char c = Character.toLowerCase(input.charAt(i));
            if (c >= 'a' && c <= 'z') {
                if ("aeiou".indexOf(c) != -1) vowels++;
                else consonants++;
            }
        }
        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
    }
}

```

Explanation: This program checks each character and counts vowels and consonants using basic conditions.

6. Find First Non-Repeated Character

```

public class FirstNonRepeated {
    public static void main(String[] args) {
        String input = "swiss";
        for (int i = 0; i < input.length(); i++) {
            char c = input.charAt(i);
            boolean isUnique = true;

```

```

        for (int j = 0; j < input.length(); j++) {
            if (i != j && c == input.charAt(j)) {
                isUnique = false;
                break;
            }
        }
        if (isUnique) {
            System.out.println("First non-repeated: " + c);
            break;
        }
    }
}

```

Explanation: This program finds the first character that does not repeat using nested loops.

7. Remove Whitespace from a String

```

public class RemoveWhitespace {
    public static void main(String[] args) {
        String input = " hello world ";
        String result = "";
        for (int i = 0; i < input.length(); i++) {
            if (input.charAt(i) != ' ') {
                result += input.charAt(i);
            }
        }
        System.out.println("Without spaces: " + result);
    }
}

```

Explanation: This program removes whitespace by skipping space characters during string construction.

8. Swap Two Strings Without Using Third Variable

```

public class SwapStrings {
    public static void main(String[] args) {
        String a = "hello";
        String b = "world";
        a = a + b;
        b = a.substring(0, a.length() - b.length());
        a = a.substring(b.length());
        System.out.println("a: " + a);
        System.out.println("b: " + b);
    }
}

```

Explanation: This program swaps two strings using concatenation and substring operations.

9. Check if Two Strings are Anagrams

```

public class AnagramCheck {
    public static void main(String[] args) {
        String s1 = "listen";
        String s2 = "silent";
        if (s1.length() != s2.length()) {

```

```

        System.out.println("Not anagrams");
        return;
    }
    int[] count = new int[256];
    for (int i = 0; i < s1.length(); i++) {
        count[s1.charAt(i)]++;
        count[s2.charAt(i)]--;
    }
    for (int i = 0; i < 256; i++) {
        if (count[i] != 0) {
            System.out.println("Not anagrams");
            return;
        }
    }
    System.out.println("Anagrams");
}
}

```

Explanation: This program uses a frequency array to compare character counts of two strings.

10. Find Longest Substring Without Repeating Characters

```

public class LongestUniqueSubstring {
    public static void main(String[] args) {
        String s = "abcabcbb";
        int maxLen = 0;
        for (int i = 0; i < s.length(); i++) {
            boolean[] seen = new boolean[256];
            int len = 0;
            for (int j = i; j < s.length(); j++) {
                char c = s.charAt(j);
                if (seen[c]) break;
                seen[c] = true;
                len++;
            }
            if (len > maxLen) maxLen = len;
        }
        System.out.println("Length: " + maxLen);
    }
}

```

Explanation: This program uses a boolean array to track seen characters and find the longest unique substring.